Project: The Credit Card Pickpocket Trap Detector

Github link: emapuff/AI4CI_project_sIAI
Team members: Magui Anca-Elena, Moisă Clementina-Ioana, Toda-Puiuleț Emanuela-Coralia

1. Project Description

Modern credit card fraud operations are distributed, high-speed, and technologically sophisticated. When bad actors acquire compromised card numbers, they orchestrate rapid, geographically distributed transactions before traditional banking systems can react.
This project proposes an end-to-end, AI-enabled simulated banking environment designed to intercept these fraudulent transactions in real-time. The system ingests two distinct data streams: a high-frequency stream of global financial transactions, and a low-frequency, asynchronous stream of customer behavioral context (e.g., GPS telemetry, device changes). An autonomous LLM-based Agent operates as the lead fraud investigator. It dynamically correlates these streams, orchestrates downstream ML models to assess immediate risk, and executes binding decisions that directly alter the simulated environment.

2. Problem definitions

P1. Correlating asynchronous streams.
Stream 1 is card transactions: thousands per second, bursty, with daily and weekly cycles.
Stream 2 is cardholder context: opt-in phone GPS pings, which are sparse, irregular, noisy and often missing, plus rare account-change events (address, phone, new device).
Events arrive late and out of order. They must be joined per card inside time windows without blocking the authorization.

P2. Latency vs. intelligence.
An authorization must be answered in about 100–300 ms. A tool-calling agent cannot sit on every transaction, so traffic must be routed.

P3. Extreme class imbalance.
Fraud is about 0.1–0.2% of transactions. Accuracy is therefore meaningless, and we need PR-AUC and recall at a fixed false-positive rate.

P4. Delayed and biased labels.
The true label arrives late: through SMS replies (minutes, or never) and chargebacks (days to weeks). For a frozen transaction we never see what would have happened, so the training data is biased by our own past decisions.

P5. Asymmetric costs in a closed loop.
Each outcome has a different cost: missed fraud loses money, an SMS adds friction, and a false freeze creates an angry customer and a support call.
Freezing stops the attacker, but the attacker then adapts. Too many false freezes flood the support queue.

P6. Drift.
Legitimate behaviour changes (holidays, travel, Black Friday). Attackers also change tactics on purpose: they slow down, use local merchants, or spoof location.

P7. Missing context is not evidence of fraud.
Location data is opt-in and imprecise (GPS off, VPNs), and online merchants have no physical location at all. Customers without location data must not be punished.
Privacy rules apply (GDPR, PCI-DSS), so we use synthetic data only.

3. State of the Art

Area	Work	What it does	Pros	Cons
Detection (Stream 1)	FICO Falcon [1]	Neural-network scoring of every authorization, trained on consortium data from many issuers	Mature; sees fraud patterns across many banks	Closed and expensive; uses transaction data only, no cardholder context
	Mastercard Decision Intelligence [2]	Network-level AI that scores each transaction against the cardholder's history	Sees every merchant on the network in real time	Proprietary; a bank cannot inspect or adapt it, or add its own data
Context (Stream 2)	Visa Mobile Location Confirmation [3]	Compares the phone's location with the shop's location	Fewer false declines when customers travel	Opt-in, privacy concerns; no help for online purchases
	GSMA CAMARA APIs [4]	Ask the mobile operator about a recent SIM swap or the device's location	Catches account takeover via SIM swap	Depends on operator coverage and consent; paid call per check
Decision (Boss AI)	Cost-sensitive decisions [5]	Decide using the money at stake per transaction, not accuracy	Simple, auditable, minimises losses	Fixed thresholds; no adaptation during an attack; ignore customer annoyance
	"Was this you?" challenge (3DS2 / SMS) [6]	Ask the customer to confirm the purchase	Fast, reliable label from the customer	Friction and abandoned purchases; OTPs can be phished
Feedback loop	Dal Pozzolo et al. [7]	Model delayed labels and investigator feedback during training	Realistic learning setup	Decisions do not change future data
	ULB transaction simulator [8]	Generates synthetic card transactions with injected fraud	Reproducible and privacy-safe	Open-loop: one stream, no adaptive attacker, decisions have no consequences
This project	Closed-loop simulator	2 correlated streams, agent choosing allow/challenge/freeze, adaptive attacker, support-flood penalty	Targets all four gaps above	Only as good as its attacker and customer models

4. Proposed Solution & Architecture Mapping

A. The Configurable Simulator & Streams

A reproducible Python-based simulation engine to generate synthetic but realistic data. It produces two distinct event streams:

· Stream 1: transactions.raw (high-speed card swipes including amounts and merchant IDs).

· Stream 2: user.context.events (asynchronous updates like device switching and GPS shifts).

Configured from a YAML file with a fixed random seed, so every run is reproducible. Besides normal cardholders, it simulates adaptive attacker agents (card testing, cash-out, account takeover) and customers who answer OTP challenges and call support when wrongly frozen.

B. Ingestion, ETL & Processing

An ingestion service validates each event against a schema, sends invalid events to a dead-letter queue, removes duplicates and normalises fields. Raw events are stored in Parquet; transactions, decisions and labels in PostgreSQL. The online processor keeps per-card state (last GPS position and its age, transaction velocity, recent device/address changes) and joins both streams per card inside time windows, tolerating late events. Offline ETL joins features with delayed labels (OTP replies, chargebacks) without future leakage, for retraining, and replays recorded days.

C. Downstream AI & Agentic Orchestration

The system utilizes two distinct AI models:

- *Downstream AI:* a gradient-boosting model (class-weighted for the ~0.1% fraud rate) scores every transaction 0-100.
- *Decision Maker (LLM Agent)*: served through an MCP server that exposes the model and context data as tools (score_transaction, get_user_history, get_location_context); the Agent chooses which to call. Missing GPS counts as "unknown", not as risk.

Routing: low scores are allowed without the Agent, mid scores go to the Agent, very high scores are frozen immediately and reviewed. Under overload, a deterministic cost-based policy decides instead.

D. Action Space & The Feedback Loop

Based on its investigation, the Agent executes one of three deterministic actions via the backend API: ALLOW, CHALLENGE_OTP, or FREEZE_CARD. The issuance of a FREEZE_CARD API call dynamically updates the simulation engine state, immediately preventing future simulated attacks on that card (Closing the loop).

For CHALLENGE_OTP, the simulated customer confirms, denies or does not reply. False freezes create support calls in a limited queue, counted as a cost. Frozen attackers move to other cards or slow down. All outcomes become labels for retraining.

E. Backend, Database & User Interface

A backend service stitches the data pipeline, the Agent, and the data warehouse together. A web dashboard provides human fraud analysts with real-time UI monitoring of the Agent's investigations and decisions.

F. Evaluation

Model level: PR-AUC, recall at a fixed false-positive rate, p95 latency. Agent and end-to-end, compared with rules-only and ML-threshold-only baselines: fraud money prevented, false-freeze rate, OTPs per 1,000 legitimate transactions, support-queue peak, cost per 1,000 transactions.

G. Stress scenario

Black Friday: 10x transaction load with holiday shopping patterns. Routing must degrade gracefully (the Agent queue falls back to the cost-based policy) without a spike in false freezes.

References

1. FICO, *Falcon Fraud Manager*, fico.com.
2. Mastercard, *Decision Intelligence* (2016) and *DI Pro* (2024).
3. Visa, *Mobile Location Confirmation*, ~2015.
4. GSMA Open Gateway / CAMARA, *SIM Swap & Device Location APIs*, 2023.
5. Correa Bahnsen et al., "Example-dependent cost-sensitive decision trees," *Expert Syst. Appl.*, 2015.
6. EMVCo, *EMV 3-D Secure Protocol 2.x*.
7. Dal Pozzolo et al., "Credit card fraud detection: a realistic modeling and a novel learning strategy," *IEEE TNNLS*, 2018.
8. Le Borgne & Bontempi, *Reproducible Machine Learning for Credit Card Fraud Detection – Practical Handbook*, ULB, 2022.